

**“Si tú no trabajas
por tus sueños,
alguien te contratará
para que trabajes
por los suyos”.**

Steve Jobs



Resolviendo Algoritmos: Pilas y Colas

Planos arquitectónicos, análisis y estrategias de solución para el
Práctico N° 2 de Estructuras de Datos.

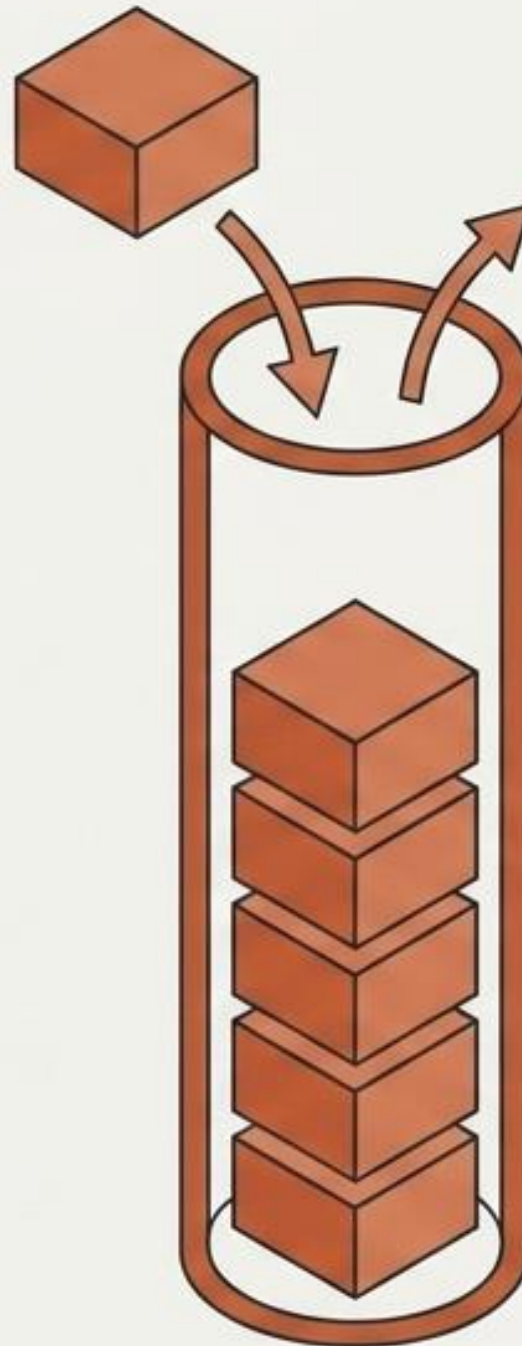
• SOBRE EL PRÁCTICO

- En este práctico se presenta una serie de **ejercicios** centrados en el estudio de las **estructuras de datos lineales**, específicamente sobre **pilas y colas**.
- Las actividades proponen la implementación técnica de estos tipos de datos abstractos utilizando representaciones **secuenciales y encadenadas**.
- Además, se incluyen problemas prácticos de simulación, como la resolución lógica de las **Torres de Hanoi** y el funcionamiento del **factorial recursivo**.
- Finalmente, se plantean escenarios del mundo real, como la gestión de una **cola de impresión** y la organización de turnos en un **sistema bancario**.
- El objetivo principal es que el estudiante analice la **eficiencia algorítmica** y la complejidad en diferentes contextos de programación.

Ejercicio 1 | Fundamentos del TDA Pila

Enunciado

Implementar el Tipo de Dato Abstracto (TDA) Pila con sus operaciones abstractas en representación secuencial y encadenada.



Last In, First Out: El último elemento en entrar es el primero en salir.

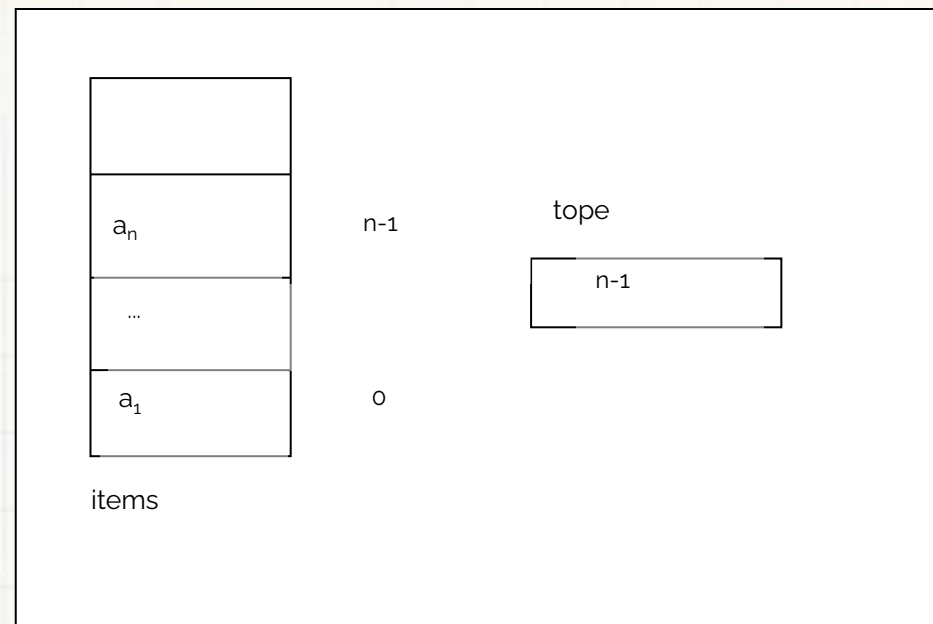
Operaciones Base

- Push(x): Apilar un elemento en la cima.
- Pop(): Desapilar y retornar el elemento de la cima.
- Top(): Leer la cima sin desapilar.
- isEmpty(): Evaluar si el contenedor carece de elementos.

Resolución Ej 1

Implementación Pila

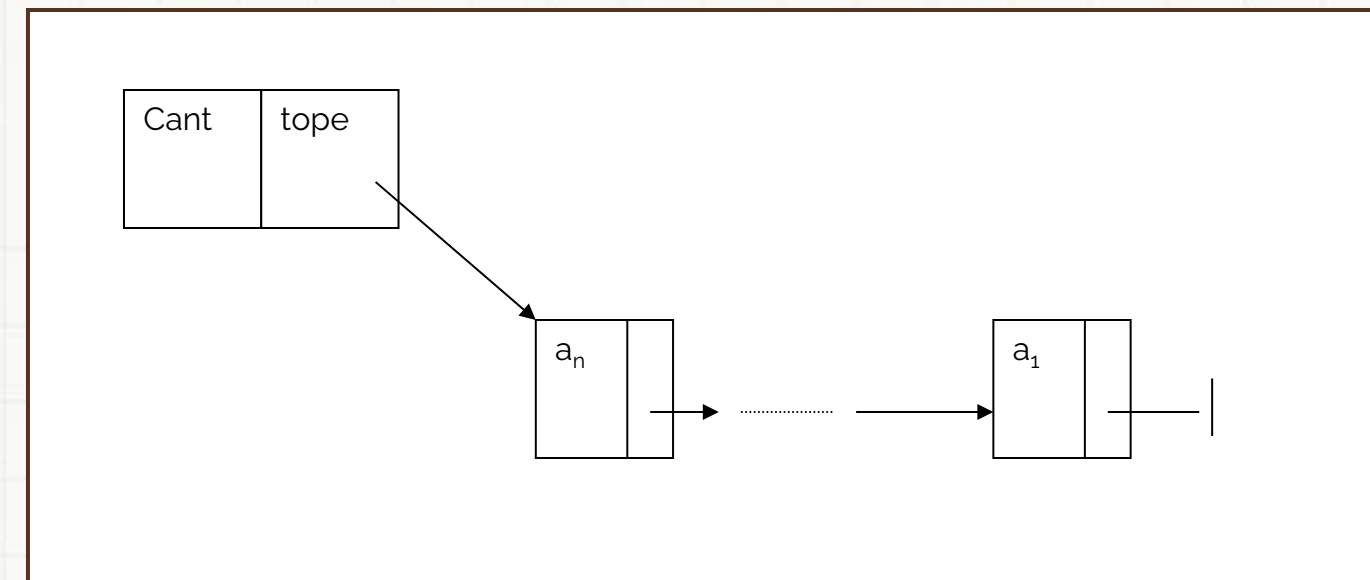
Representación Secuencial (Arreglos)



Estructura: Arreglo estático de tamaño N + Variable entera Tope.

- ✓ **Ventaja:** Acceso en tiempo constante $O(1)$ sin sobrecarga de punteros.
- ⚠ **Riesgo:** Límite físico estricto. Riesgo de Overflow si $Tope == N$.

Representación Encadenada (Nodos)



Estructura: Nodos dinámicos enlazados + Puntero Cabeza.

- ✓ **Ventaja:** Crecimiento dinámico ilimitado (hasta agotar RAM).
- ⚠ **Riesgo:** Fragmentación de memoria y overhead espacial por almacenar el puntero al Siguiente en cada nodo.

REPRESENTACIÓN SECUENCIAL

```
class pila
{   int *items;
    int tope;
    int cant;
    public:
    pila(int xcant=0):
    cant(xcant)
    {   tope=-1;
        items=new int[cant];
    }
    int vacía(void)
    {   return (tope== -1);
    }
    int insertar(int x)
    {
    if (tope<cant-1)
        { items[++tope]=x;
          return (x);
        }
    else return (0);
    }
```

```
int suprimir(void)
{   int x;
    if (vacía())
    {   printf("%s","Pila vacía");
        return(0);
    }
    else
    {   x=items[tope--];
        return(x);
    }
}

void mostrar(void)
{ int i;
  if (!vacía())
  {   for (i=tope; i>=0; i--)
      {   cout<<items[i]<<endl;
          }
      }
  };
```

REPRESENTACIÓN EN CADENA

```
class celda
{
    int item;
    celda *sig;
public:
    int obteneritem(void)
    {
        return(item);
    }
    void cargaritem(int xitem)
    {
        item=xitem;
    }
    void cargarsig(celda* xtope)
    {
        sig=xtope;
    }
    celda* obtenersig(void)
    {
        return(sig);
    }
};
```

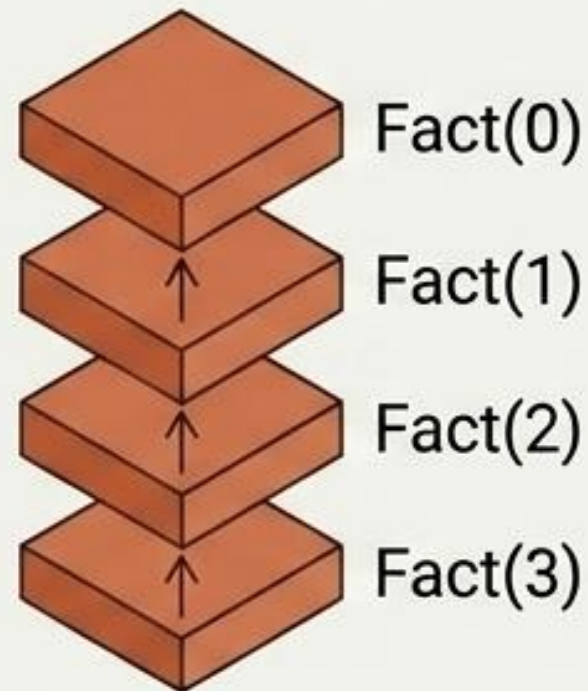
```
class pila
{
    int cant;
    celda *tope;
public:
    pila(celda* xtope=NULL,int
xcant=0):
    tope(xtope),cant(xcant)
    { }
    int vacía(void)
    {
        return (cant==0);
    }
    int insertar(int x)
    {
        celda *ps1;
        ps1=new(celda);
        ps1->cargaritem(x);
        ps1->cargarsig(tope);
        tope=ps1;
        cant++;
        return(ps1->obteneritem());
    }
```

```
int suprimir(void)
{ celda *aux;
  int x;
  if (vacía())
      { printf("%s","Pila vacía");
        return(0);
      }
  else
      { aux=tope;
        x=tope->obteneritem();
        tope=tope->obtenersig();
        cant--;
        free(aux);
        return(x);
      }
}
```

Ejercicio 2 | El Stack de Recursión (Factorial)

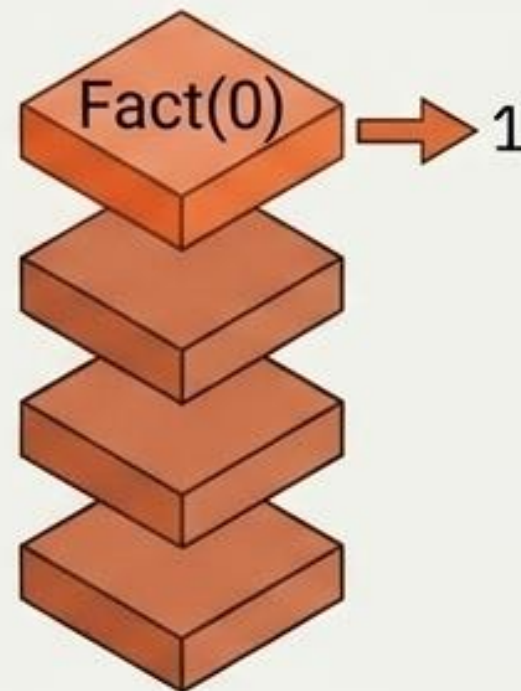
Enunciado: Simular el stack de recursión para la función:
 $n! = n * (n - 1)!$ si $n > 0$, y $n! = 1$ si $n = 0$.

Apilamiento (Winding)



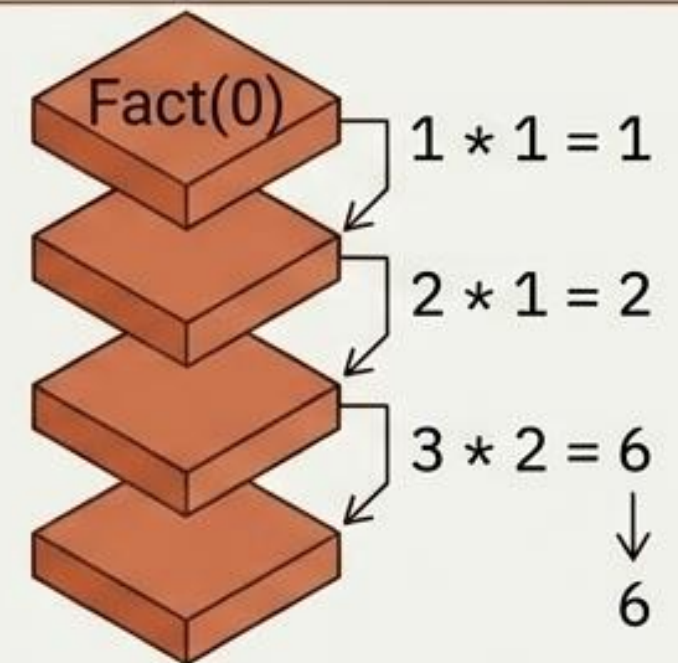
Las llamadas se apilan suspendiendo su ejecución hasta alcanzar el caso base.

Caso Base



Se alcanza $n=0$. El tope de la pila resuelve su valor matemático (1).

Desapilamiento (Unwinding)

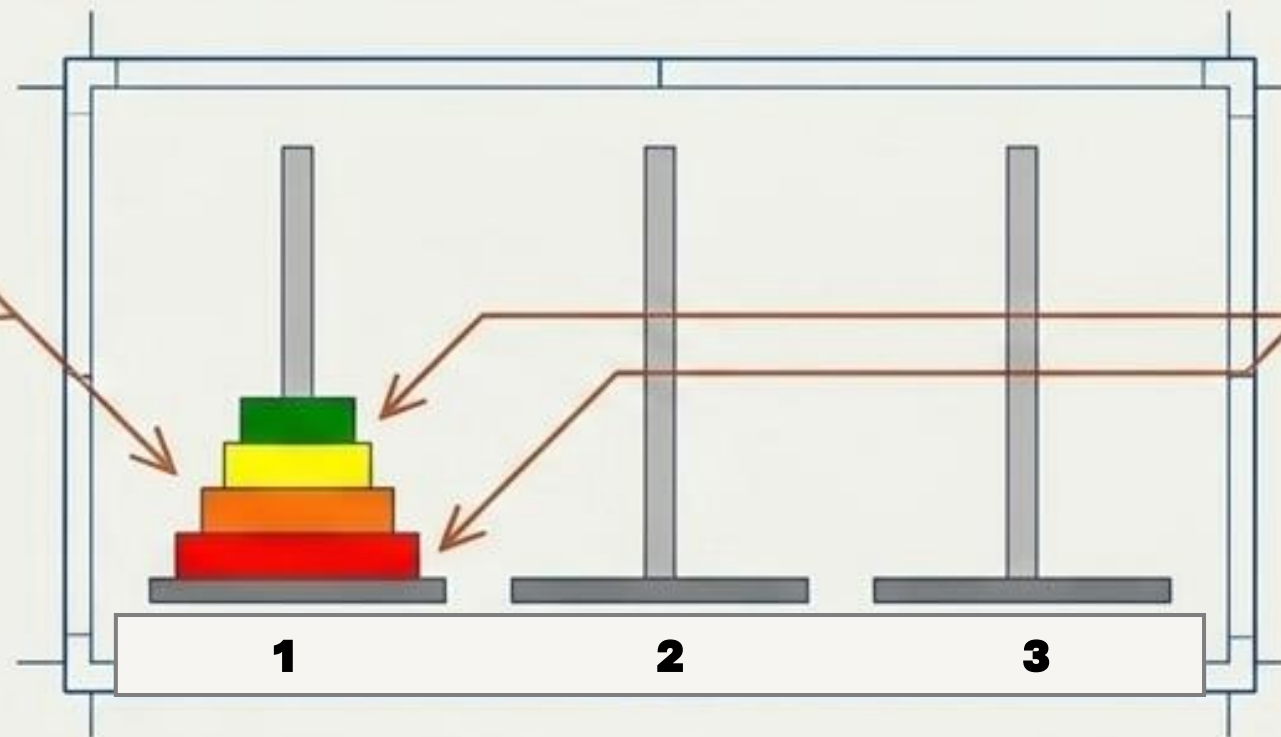


El sistema realiza un Pop sucesivo. Cada bloque caído multiplica su n por el resultado del bloque anterior hasta retornar el total (6).

Ejercicio 3 | Análisis: Torres de Hanoi

Enunciado: Simular el juego trasladando n discos de la Pila 1 a la Pila 3 usando la Pila 2 como auxiliar. Reportar jugadas vs. el mínimo teórico.

Regla 1 (Física):
Sólo se puede mover una pieza a la vez. No se pueden retener piezas piezas en memoria.



Regla 2 (Lógica):
Un disco mayor NUNCA puede apilarse sobre un disco menor.

Condición de Victoria:

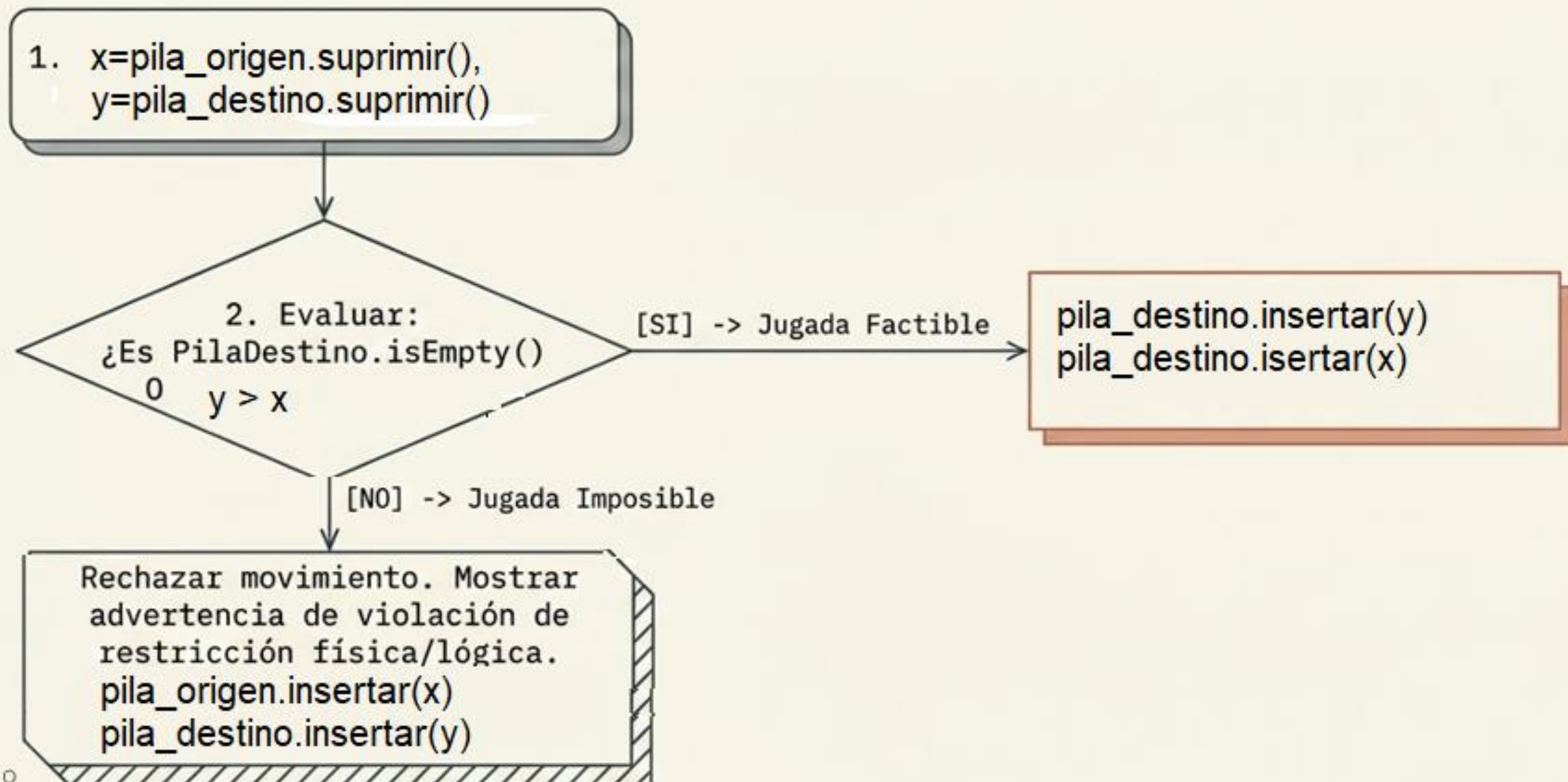
`Pila 1 y 2 == isEmpty().`

Métrica Objetivo:

Alcanzar la solución en $2^n - 1$ movimientos.

Resolución Ej 3 | Arquitectura del Motor de Juego

Estructura de Datos Base: Tres instancias del TDA Pila independientes (Origen, Auxiliar, Destino). Discos representados por números enteros.





**LA ÚNICA FORMA DE HACER
UN GRAN TRABAJO, ES AMAR
LO QUE HACES.**

STEVE JOBS

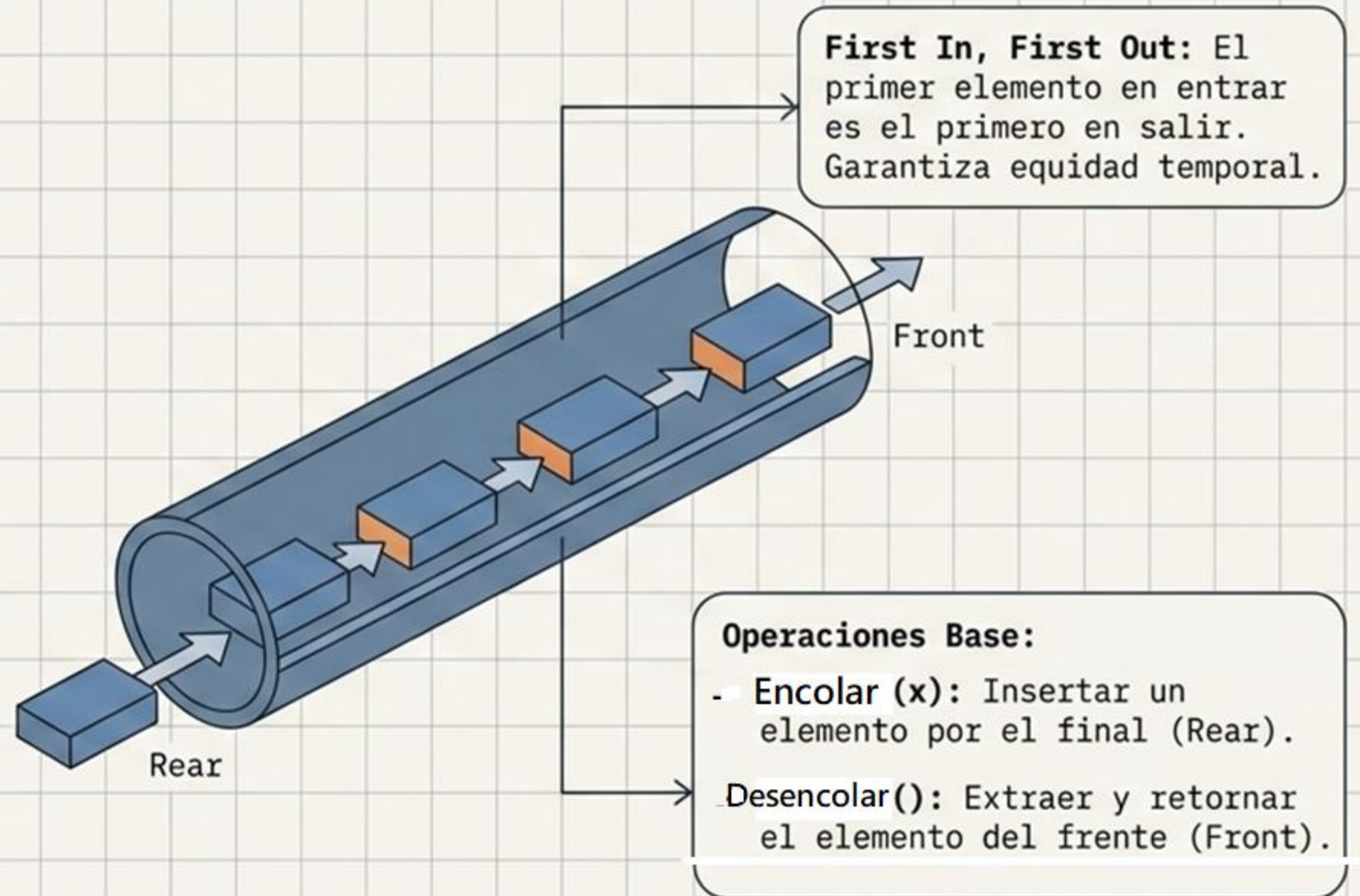
DUPAO



Ejercicio 4 | Fundamentos del TDA Cola

Enunciado

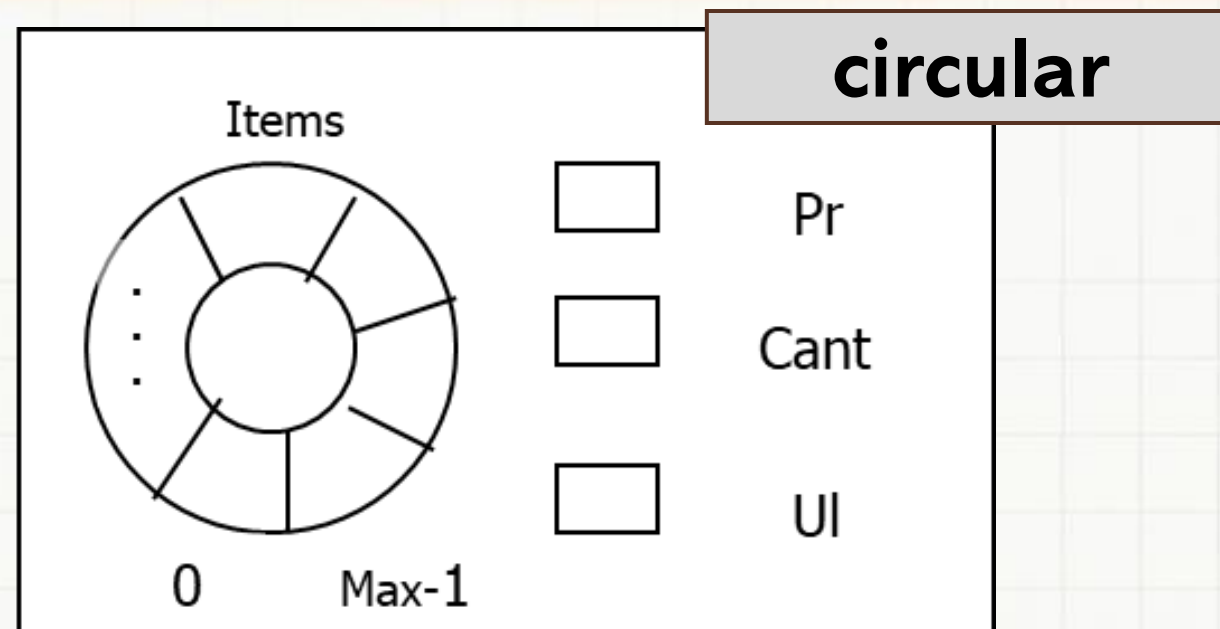
Implementar el Tipo de Dato Abstracto (TDA) Cola con sus operaciones abstractas en representación secuencial y encadenada.



En diferentes bibliografías pueden encontrar:
Encolar() o Insertar()
Desencolar() o suprimir()

Ejercicio 4 Implementación TDA COLA

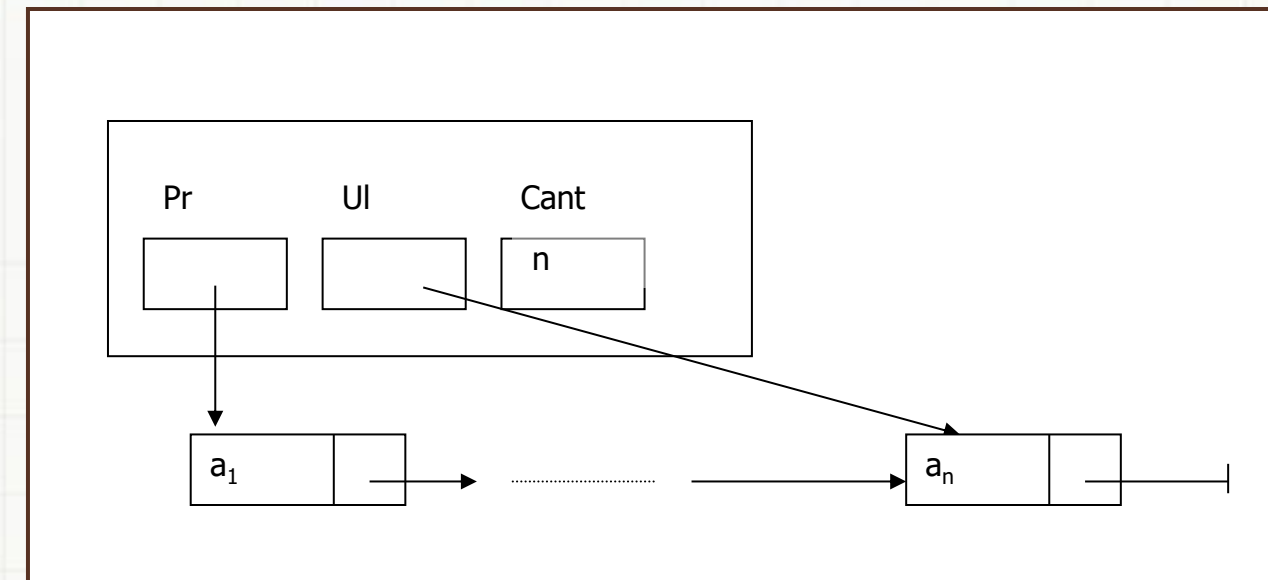
Representación Secuencial (Arreglos)



Estructura: Arreglo estático de tamaño $N + 1$. Primero y Ultimo indican donde insertar y suprimir.

- ✓ **Ventaja:** Acceso en tiempo constante $O(1)$ sin sobrecarga de punteros.
- ⚠ **Riesgo:** Límite físico estricto. Riesgo de Overflow si $Tope == N$.

Representación Encadenada (Nodos)



Estructura: Nodos dinámicos enlazados + dos punteros: Primero y Ultimo indican donde insertar y suprimir.

- ✓ **Ventaja:** Crecimiento dinámico ilimitado (hasta agotar RAM).
- ⚠ **Riesgo:** Fragmentación de memoria y overhead espacial por almacenar el puntero al Siguiente en cada nodo.

REPRESNTACION SECUENCIAL

```
class cola
{  int *items;
  int pr;
  int ul;
  int cant;
  int max;
public:
  cola(int xmax=0):
  max(xmax)
  {  pr=0;
    ul=0;
    cant=0;
    items=new int[max];
  }
  int vacía(void)
  {  return (cant==0);  }
  int insertar(int x)
  {  if (cant<max)
      {  items[ul]=x;
        ul=(ul+1)%max;
        cant++;
        return (x);
      }
    else return (0);
  }
```

```
int suprimir(void)
{  int x;
  if (vacía())
  {  printf("%s","Pila vacía");
    return(0);
  }
  else
  {  x=items[pr];
    pr=(pr+1)%max;
    cant--;
    return(x);
  }
}

void recorrer(void)
{  int i,j;
  if (!vacía())
  {  i=pr;
    j=0;
    for (i; j<cant ; i=(i+1)%max,j++)
    {  cout<<items[i]<<endl;
      }
    }
  }
};
```

REPRESENTACIÓN

ENCADENA

class celda

```
{
    int item;
    celda *sig;
public:
    int obteneritem(void)
    {
        return(item);
    }
    void cargaritem(int xitem)
    {
        item=xitem;
    }
    void cargarsig(celda* xtope)
    {
        sig=xtope;
    }
    celda* obtenersig(void)
    {
        return(sig);
    }
};
```

class cola

```
{ int cant;
    celda *pr;
    celda *ul;
public:
    cola(celda* xpr=NULL,celda
*xul=NULL,int xcant=0):
    pr(xpr),ul(xul),cant(xcant)
    {}
    int vacía(void)
    { return (cant==0);
    }
    int insertar(int x)
    { celda *ps1;
      ps1=new(celda);
      ps1->cargaritem(x);
      ps1->cargarsig(NULL);
      if (pr==NULL)
          {pr=ps1;}
      else
          {ul->cargarsig(ps1);}
      ul=ps1;
      cant++;
      return(ul->obteneritem());
    }
}
```

int **suprimir**(void)

```
{ celda *aux;
  int x;
  if (vacía())
      { printf("%s","Cola vacía");
        return(0);
      }
  else
      { aux=pr;
        x=pr->obteneritem();
        pr=pr->obtenersig();
        cant--;
        free(aux);
        return(x);
      }
  }
celda* recuperapr(void)
{
    return(pr);
}
void recorrer(celda *aux)
{
    if (aux!=NULL)
        { cout<<aux->obteneritem()<<endl;
          recorrer(aux->obtenersig());
        }
    }
};
```

Ingresar : Frecuencia de Llegada de Clientes (x),
Tiempo de atención de Cajero (y) ,Tiempo máximo de Simulación (TMS).

Cada cuanto llega un cliente(promedio)

Inicializar Reloj

MIENTRAS (Reloj no alcance el TMS) **HACER**

SI (llega Cliente)

1 / x

ENTONCES Insertar(Cola, Cliente)

FIN SI

SI (Cajero libre)

ENTONCES

SI (No (Vacía(Cola)))

ENTONCES

Suprimir (Cola, Cliente)

Computar el Tiempo de espera del Cliente en la Cola

Acumular Tiempo de espera del Cliente

Contar cliente atendido

Inicializar Cajero Ocupado

cajero= 0

Cajero ocupado ← y

FIN SI

FIN SI

Actualizar Reloj

SI (Cajero Ocupado)

ENTONCES Actualizar Cajero

FIN SI

FIN MIENTRAS

Tiempo Promedio de espera = $\frac{\text{Tiempo de espera Acumulado}}{\text{Cantidad de Clientes Atendidos}}$

Reportar (Tiempo Promedio de espera)



Llega cliente: número aleatorio (random) entre 0 y 1/ Frecuencia de Llegada de un cliente

Cuando el cliente llega, ingresa a la Cola (Reloj o 0)

Cajero ocupado/ libre: tiempo atención cajero y minutos

Cajero = 0 →

Cajero libre

Cajero > 0 →

Cajero Ocupado

Inicializar Cajero Ocupado: 0 o Tiempo de atención de cajero

Para controlar el Tiempo máximo de Simulación

• Reloj = 0 →

Inicio simulación

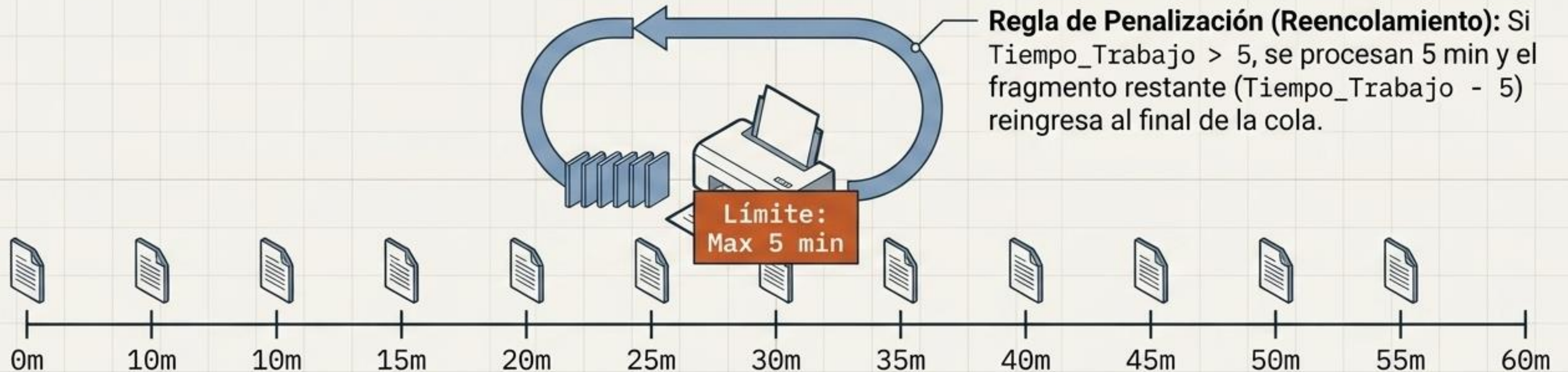
• Reloj = Tiempo Máximo Simulación Simulación

→

Fin

Ejercicio 5 | Simulación de Impresión (Time-Slicing)

Enunciado: Simular una cola de impresión por 1 hora. Llegan trabajos cada 5 min. Tiempo máximo de proceso: 5 min. Trabajos inacabados vuelven a la cola.

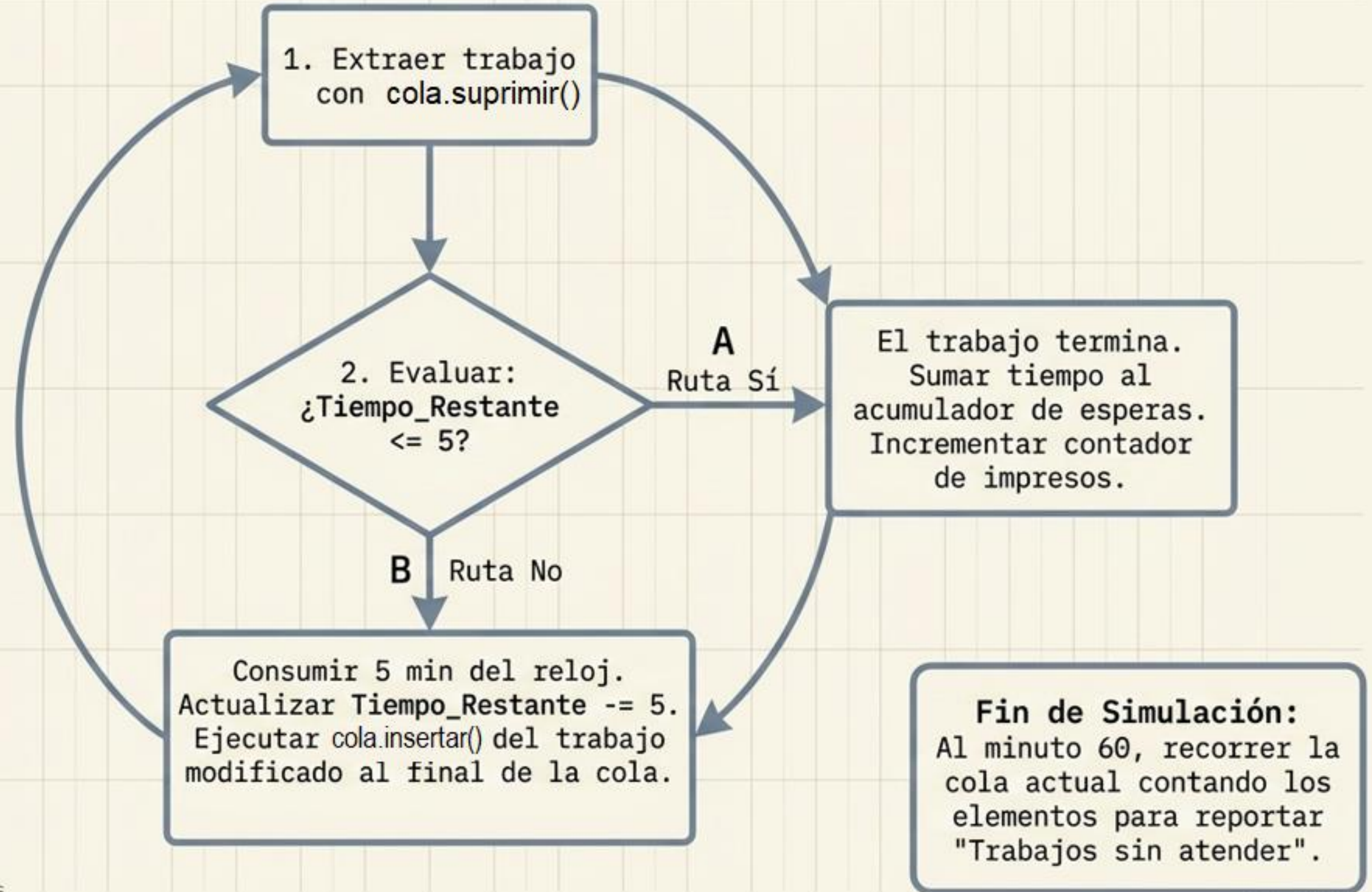


Métricas a Reportar:

- Trabajos sin atender.
- Promedio de espera de impresos.

Resolución Ej 5 | Algoritmo de Reencolamiento (Round-Robin)

Estructura Base:
Cola FIFO única
(ColaImpresion).



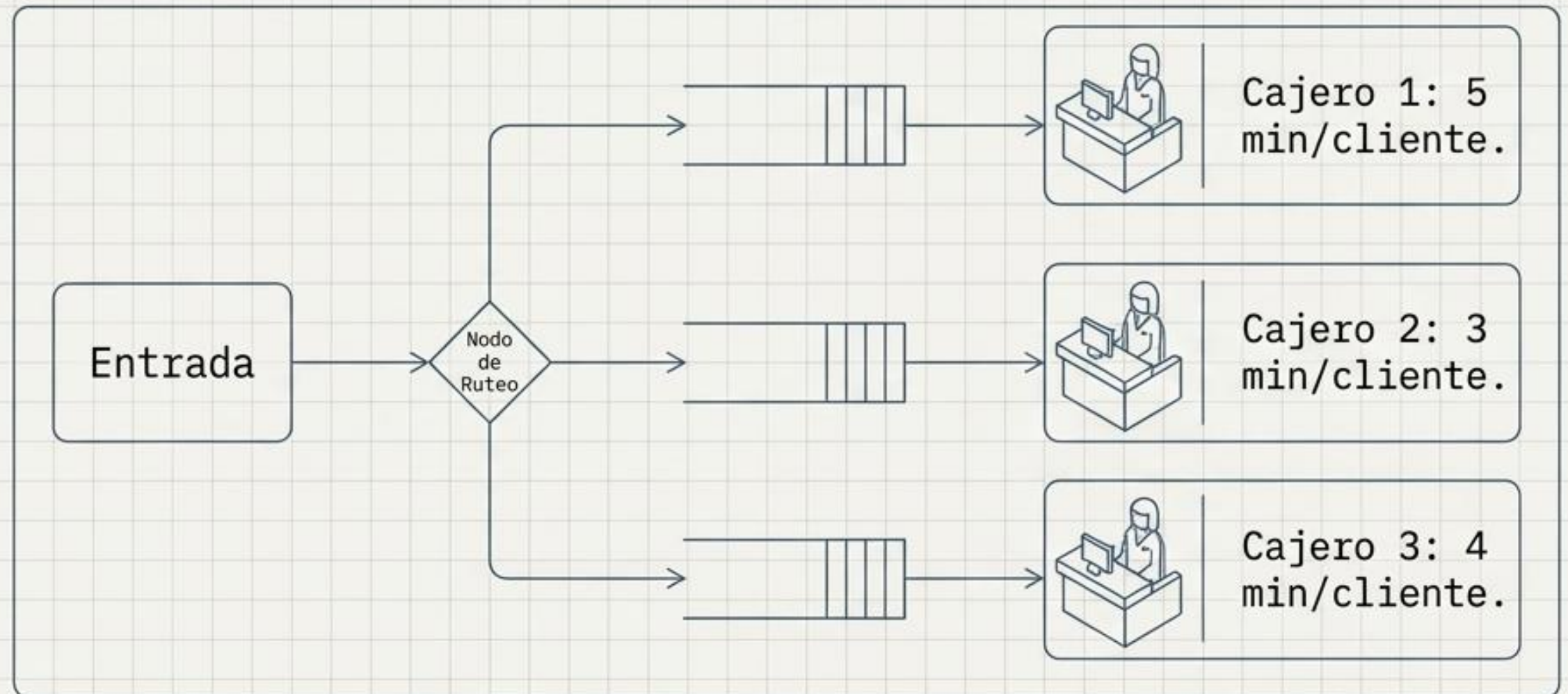
Ejercicio 6 | Simulación Bancaria Multicola

Parámetros de Simulación:

- Tiempo total: 2 horas (120 minutos).
- Llegadas: 1 cliente cada 2 minutos.

Métricas:

- Máx espera,
- Atendidos,
- Sin atender,
- Promedio espera.

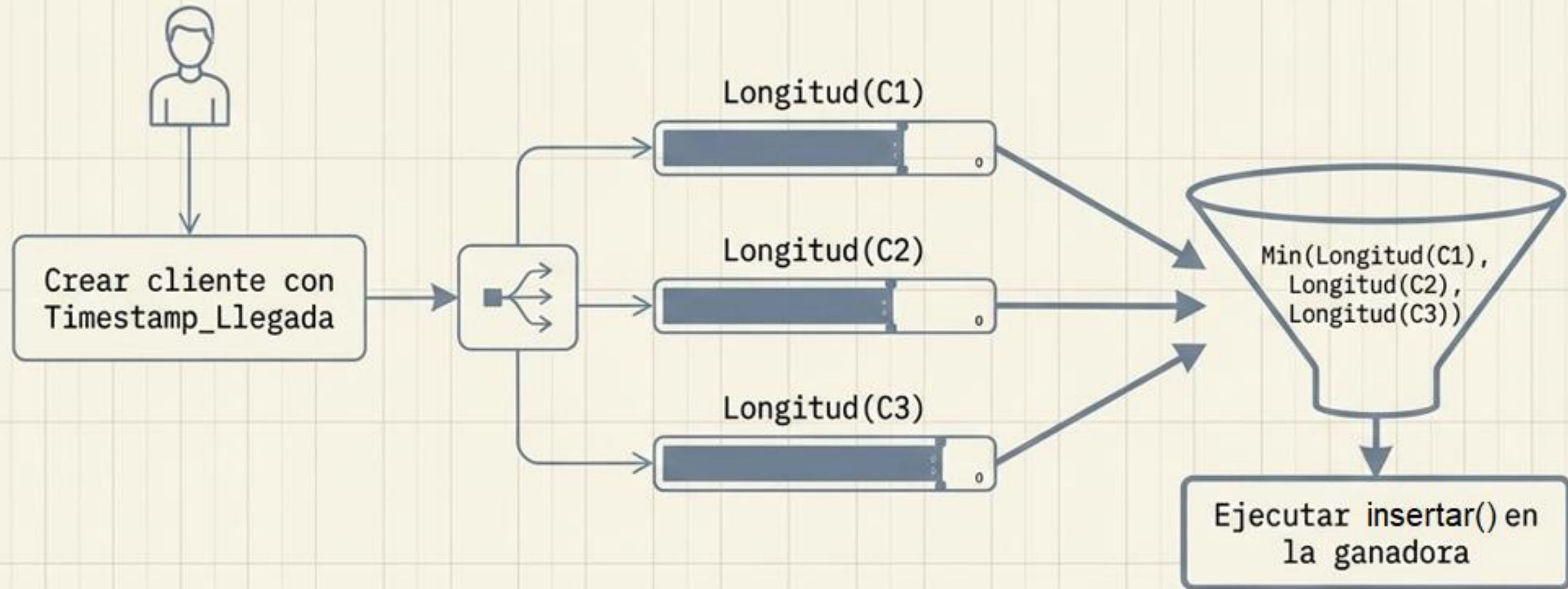


Reglas de Enrutamiento de Clientes:

1. Si hay cajero libre -> Elección Aleatoria.
2. Si todos ocupados -> Cola más corta.
3. Si hay empate en cola corta -> Elección Aleatoria.

Resolución Ej 6 | Enrutador Heurístico de Clientes

Arquitectura de Datos: Tres instancias de TDA Cola (C1, C2, C3) ampliadas con una función Longitud() que retorna su tamaño actual.



Motor de Simulación (Procesamiento)

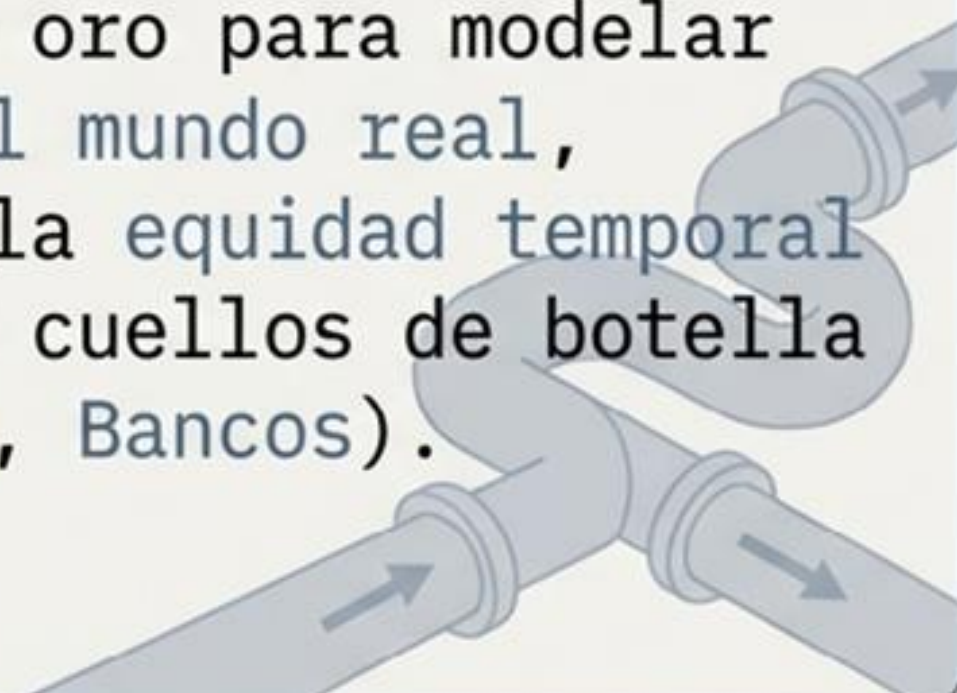
Si la cola no está vacía, restar 1 tick al tiempo de atención del cliente en el `Front()`.
Si llega a 0, `suprimir()` y calcular tiempos de espera (`Tick_Actual - Timestamp_Llegada`).

Conclusión | El Plano de los Datos

Pilas (LIFO): Arquitecturas para controlar el pasado. Son esenciales para rastrear el **historial** de **acciones**, volver sobre nuestros pasos y resolver problemas de profundidad lógica (**Recursividad, Hanoi, laberintos, etc**)



Colas (FIFO): Arquitecturas para gestionar el futuro. Son el estándar de oro para modelar sistemas del mundo real, garantizar la **equidad temporal** y optimizar cuellos de botella (Impresoras, Bancos).



La elección del TDA correcto no es un detalle de implementación; es la arquitectura subyacente que define si un sistema falla por desbordamiento o fluye con eficiencia.